



Personal Library Tracker — Full Technical Specification

Purpose: A comprehensive Cursor-ready specification for building a personal library tracking web application with a public catalog view and a private admin dashboard rich with statistics and management tools.

Stack: React (Vite + TypeScript) · Node.js / Express · PostgreSQL · Railway hosting

Audience for this doc: AI code-generation tools (Cursor, Copilot) and developers.

Table of Contents

1. [Project Overview](#)
 2. [Tech Stack & Tooling](#)
 3. [Database Schema](#)
 4. [API Design \(REST\)](#)
 5. [Authentication & Authorization](#)
 6. [Frontend Architecture](#)
 7. [Public Catalog \(Viewer Page\)](#)
 8. [Admin Dashboard](#)
 9. [Book Data Model & Business Logic](#)
 10. [CSV Import Feature](#)
 11. [Statistics & Analytics \(Admin\)](#)
 12. [UI/UX Design Guidelines](#)
 13. [Railway Deployment Guide](#)
 14. [Seed Data Reference](#)
 15. [Future Enhancements \(Optional\)](#)
-

1. Project Overview

What is this?

A **personal library management web app** that lets the owner (admin) track every book they own or plan to read — including purchase price, market price, savings, reading status, book format (physical / digital / audio), editions, authors, page counts, publishers, ISBNs, and more.

Two User Roles

Role	Access	Capabilities
Viewer (public, no login)	/ public catalog	Browse books, search, filter. Only sees books the admin has marked as publicly visible .
Admin (single user, password-protected)	/admin/*	Full CRUD on all books, toggle public visibility per book, CSV import, rich statistics dashboard, manage authors/publishers/shelves.

Key Features

- **3 book formats:** Physical, Digital (Kindle/ePub), Audiobook — each book has exactly one format.
 - **Savings tracking:** $\text{savings} = \text{purchase_price} - \text{market_price}$ (market price a.k.a. “Actual Price”).
 - **Public visibility toggle:** Admin can show/hide any book from the public catalog.
 - **Rich admin statistics:** Spending breakdowns, reading progress, author/publisher analytics, format distribution, timeline charts, and more.
 - **CSV import:** Bulk-import books from a CSV matching the owner’s existing Goodreads-style export format.
 - **Responsive design:** Works on desktop and mobile.
-

2. Tech Stack & Tooling

Frontend

Tool	Purpose
React 18+	UI framework
Vite	Build tool & dev server
TypeScript	Type safety throughout
React Router v6	Client-side routing
TanStack Query (React Query)	Server state management, caching, pagination
Recharts or Chart.js (via react-chartjs-2)	Statistics charts & graphs
Tailwind CSS	Utility-first styling
Shadcn/ui	Pre-built accessible component library (built on Radix UI)
Lucide React	Icon set
react-hot-toast	Toast notifications
papaparse	Client-side CSV parsing (for import preview)

Backend

Tool	Purpose
Node.js 20+	Runtime
Express.js	HTTP server & routing
TypeScript	Type safety
Prisma ORM	Database access, migrations, type-safe queries
PostgreSQL 15+	Primary database
bcrypt	Password hashing
jsonwebtoken (JWT)	Admin session tokens
multer	File upload handling (CSV import)
csv-parser or papaparse	Server-side CSV parsing
cors	Cross-origin resource sharing
helmet	Security headers
express-rate-limit	Rate limiting
zod	Request validation

Project Structure

```

library-tracker/
├── client/                                     # React frontend (Vite)
│   ├── src/
│   │   ├── components/
│   │   │   ├── ui/                          # shadcn/ui components
│   │   │   ├── layout/                     # Header, Footer, Sidebar
│   │   │   ├── books/                     # BookCard, BookList, BookDetail, BookForm
│   │   │   ├── admin/                    # Dashboard widgets, StatsCards
│   │   │   └── common/                   # SearchBar, FilterPanel, Pagination
│   │   ├── pages/
│   │   │   ├── public/                  # CatalogPage, BookDetailPage
│   │   │   └── admin/                  # LoginPage, DashboardPage, BooksManagePage, Import-
│   │   └── Page
│   │       ├── hooks/                  # useBooks, useStats, useAuth, etc.
│   │       ├── lib/                   # api client, utils, constants
│   │       ├── types/
│   │       └── App.tsx                # TypeScript interfaces
│   ├── index.html
│   ├── tailwind.config.ts
│   ├── vite.config.ts
│   └── package.json
├── server/                                   # Express backend
│   ├── src/
│   │   ├── routes/                    # books.ts, authors.ts, stats.ts, auth.ts, import.ts
│   │   ├── middleware/               # auth.ts, errorHandler.ts, rateLimiter.ts
│   │   ├── services/                 # bookService.ts, statsService.ts, importService.ts
│   │   ├── validators/
│   │   ├── utils/
│   │   └── index.ts                  # zod schemas
│   │                                   # helpers
│   └── prisma/                       # Express app entry
│       ├── schema.prisma
│       ├── seed.ts
│       └── migrations/
│       └── package.json
├── .env.example
├── railway.toml
└── README.md

```

3. Database Schema

Prisma Schema (`server/prisma/schema.prisma`)

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ===== ENUMS =====

enum BookFormat {
  PHYSICAL
  DIGITAL
  AUDIO
}

enum ReadingStatus {
  TO_READ
  READING
  READ
  DID_NOT_FINISH
  ON_HOLD
}

enum BindingType {
  PAPERBACK
  HARDCOVER
  MASS_MARKET_PAPERBACK
  KINDLE_EDITION
  UNKNOWN
}

// ===== MODELS =====

model Admin {
  id          String @id @default(cuid())
  username    String @unique
  passwordHash String
  createdAt   DateTime @default(now())
}

model Author {
  id          String @id @default(cuid())
  name        String @unique
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

// Relations
booksAsPrimary Book[] @relation("PrimaryAuthor")
booksAsAdditional BookAdditionalAuthor[]

model Publisher {
  id          String @id @default(cuid())
  name        String @unique
  createdAt   DateTime @default(now())

  books       Book[]
}

model Bookshelf {

```

```

    id          String    @id @default(cuid())
    name        String    @unique
    createdAt   DateTime  @default(now())

    books       BookBookshelf[]

  }

model Book {
  id          String    @id @default(cuid())
  externalId  String?   @unique // "Book Id" from CSV / Goodreads
  title       String
  isbn        String?
  isbn13      String?

  // Pricing
  purchasePrice Decimal? @db.Decimal(10, 2)
  marketPrice  Decimal? @db.Decimal(10, 2) // "Actual Price"
  currency      String    @default("SAR")

  // Book metadata
  format        BookFormat @default(PHYSICAL)
  binding       BindingType @default(PAPERBACK)
  numberOfPages Int?
  yearPublished Int?
  originalPublicationYear Int?
  edition       String?    // free-text edition info e.g. "1st Edition",
  "Revised 2020"

  // Status
  status        ReadingStatus @default(TO_READ)
  dateAdded     DateTime      @default(now())
  dateStartedReading DateTime?
  dateFinishedReading DateTime?

  // Visibility
  isPubliclyVisible Boolean @default(true)

  // Cover image
  coverImageUrl String?

  // Notes (admin only)
  notes String?

  // Timestamps
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  // Relations
  authorId    String
  author      Author    @relation("PrimaryAuthor", fields:
[authorId], references: [id])
  additionalAuthors BookAdditionalAuthor[]
  publisherId String?
  publisher    Publisher? @relation(fields: [publisherId], references:
[id])
  bookshelves BookBookshelf[]

  @@index([title])
  @@index([status])
  @@index([format])
  @@index([isPubliclyVisible])
  @@index([authorId])
  @@index([publisherId])

```

```

model BookAdditionalAuthor {
  bookId      String
  authorId    String
  book        Book    @relation(fields: [bookId], references: [id], onDelete: Cascade)
  author      Author  @relation(fields: [authorId], references: [id], onDelete: Cascade)

  @@id([bookId, authorId])
}

model BookBookshelf {
  bookId      String
  bookshelfId String
  book        Book    @relation(fields: [bookId], references: [id], onDelete: Cascade)
  bookshelf   Bookshelf @relation(fields: [bookshelfId], references: [id], onDelete: Cascade)

  @@id([bookId, bookshelfId])
}

```

Key Schema Decisions

Decision	Rationale
Separate <code>Author</code> table	Many books share authors — normalize to avoid duplication and enable author-level analytics.
<code>BookAdditionalAuthor</code> join table	Handles “Additional Authors” / translators as a many-to-many relationship.
Separate <code>Publisher</code> table	Enables publisher-level analytics and prevents typo duplicates.
<code>Bookshelf</code> as tags (many-to-many)	Bookshelves like “to-read”, “أدب”, “روايات” are tags; a book can belong to multiple.
<code>edition</code> as free text	Editions vary wildly (“1st”, “Revised 2020”, “Collector’s”); free text is more practical than a separate table.
<code>format</code> enum for Physical/Digital/Audio	Core requirement — exactly one format per book record.
Computed <code>savings</code>	Not stored in DB — computed on the fly as <code>purchasePrice - marketPrice</code> . This avoids stale data.
<code>currency</code> field	Defaults to “SAR” (Saudi Riyal) based on the data, but allows flexibility.

4. API Design (REST)

Base URL: `/api`

Public Endpoints (no auth)

GET	<code>/api/books</code>	# List publicly visible books (paginated, filterable)
GET	<code>/api/books/:id</code>	# Single book detail (only if publicly visible)
GET	<code>/api/authors</code>	# List all authors (that have public books)
GET	<code>/api/publishers</code>	# List all publishers (that have public books)
GET	<code>/api/bookshelves</code>	# List all bookshelves

Auth Endpoints

POST	<code>/api/auth/login</code>	# Admin login → returns JWT
POST	<code>/api/auth/change-password</code>	# Change admin password (auth required)
GET	<code>/api/auth/me</code>	# Validate current token

Admin Endpoints (JWT required)

```
# Books CRUD
GET    /api/admin/books
# List ALL books (including hidden, paginated, filterable)
GET    /api/admin/books/:id      # Single book detail (any book)
POST   /api/admin/books          # Create new book
PUT     /api/admin/books/:id     # Update book
DELETE  /api/admin/books/:id     # Delete book
PATCH  /api/admin/books/:id/visibility # Toggle public visibility

# Bulk operations
PATCH  /api/admin/books/bulk-visibility # Toggle visibility for multiple books
DELETE  /api/admin/books/bulk-delete     # Delete multiple books

# Authors
GET     /api/admin/authors          # List all authors with book counts
POST    /api/admin/authors          # Create author
PUT     /api/admin/authors/:id      # Update author
DELETE  /api/admin/authors/:id      # Delete author (fails if has books)

# Publishers
GET     /api/admin/publishers       # List all publishers with book counts
POST    /api/admin/publishers       # Create publisher
PUT     /api/admin/publishers/:id   # Update publisher
DELETE  /api/admin/publishers/:id   # Delete publisher (fails if has books)

# Bookshelves
GET     /api/admin/bookshelves      # List all bookshelves with book counts
POST    /api/admin/bookshelves      # Create bookshelf
PUT     /api/admin/bookshelves/:id  # Update bookshelf
DELETE  /api/admin/bookshelves/:id  # Delete bookshelf

# CSV Import
POST    /api/admin/import/csv       # Upload & import CSV file
POST    /api/admin/import/preview   # Upload CSV and return preview (dry run)

# Statistics
GET     /api/admin/stats/overview   # High-level KPIs
GET     /api/admin/stats/spending   # Spending analytics
GET     /api/admin/stats/reading    # Reading progress analytics
GET     /api/admin/stats/authors    # Author-level analytics
GET     /api/admin/stats/publishers # Publisher-level analytics
GET     /api/admin/stats/formats    # Format distribution
GET     /api/admin/stats/timeline   # Books added over time
GET     /api/admin/stats/bookshelves # Bookshelf distribution
```

Query Parameters for Book Listing

All book list endpoints (`GET /api/books` and `GET /api/admin/books`) support:

Parameter	Type	Description	Example
page	number	Page number (1-based)	?page=2
limit	number	Items per page (default 20, max 100)	?limit=50
search	string	Full-text search on title, author name, ISBN	?search=مارکیز
format	enum	Filter by book format	?format=PHYSICAL
status	enum	Filter by reading status	?status=READ
binding	enum	Filter by binding type	?binding=PAPERBACK
authorId	string	Filter by author	?authorId=xxx
publisherId	string	Filter by publisher	?publisherId=xxx
bookshelfId	string	Filter by bookshelf	?bookshelfId=xxx
minPrice	number	Minimum purchase price	?minPrice=20
maxPrice	number	Maximum purchase price	?maxPrice=100
minPages	number	Minimum page count	?minPages=200
maxPages	number	Maximum page count	?maxPages=500
yearFrom	number	Published year range start	?yearFrom=2000
yearTo	number	Published year range end	?yearTo=2024
sortBy	string	Sort field: title , purchasePrice , numberOfPages , dateAdded , yearPublished	?sortBy=purchasePrice
sortOrder	asc / desc	Sort direction	?sortOrder=desc

Response Format

All API responses follow this structure:

```
// Success (single item)
{
  "success": true,
  "data": { /* book/author/etc object */ }
}

// Success (list with pagination)
{
  "success": true,
  "data": [ /* array of items */ ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "totalItems": 271,
    "totalPages": 14
  }
}

// Error
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Title is required",
    "details": [ /* optional field-level errors */ ]
  }
}
```

5. Authentication & Authorization

Single Admin User

This app has **one admin user only** (the library owner). No registration flow — the admin account is seeded.

Implementation

1. **Seed the admin:** During `prisma db seed`, create the admin user with a hashed password from the `ADMIN_PASSWORD` environment variable.
2. **Login flow:**
 - `POST /api/auth/login` with `{ username, password }`.
 - Validate credentials with `bcrypt.compare()`.
 - Return a JWT token (expires in 7 days) containing `{ adminId, username }`.
 - Store the token in `localStorage` on the client.
3. **Route protection:**
 - Backend: Express middleware that verifies the JWT on all `/api/admin/*` routes.
 - Frontend: `<ProtectedRoute>` wrapper that redirects to `/admin/login` if no valid token.
4. **Environment variables:**

```
env
```

```
ADMIN_USERNAME=admin
```

```
ADMIN_PASSWORD=your-secure-password-here
```

```
JWT_SECRET=your-jwt-secret-here
```

6. Frontend Architecture

Routing Map

```
<Routes>
  { /* — PUBLIC — */ }
  <Route path="/" element={<PublicLayout />} >
    <Route index element={<CatalogPage />} />
    <Route path="books/:id" element={<PublicBookDetailPage />} />
  </Route>

  { /* — ADMIN — */ }
  <Route path="/admin/login" element={<AdminLoginPage />} />
  <Route path="/admin" element={<ProtectedRoute><AdminLayout /></ProtectedRoute>} >
    <Route index element={<DashboardPage />} />           { /* Statistics home */ }
    <Route path="books" element={<BooksManagePage />} />   { /* Full book list +
CRUD */ }
    <Route path="books/new" element={<BookFormPage />} />   { /* Add book */ }
    <Route path="books/:id/edit" element={<BookFormPage />} /> { /* Edit book */ }
    <Route path="authors" element={<AuthorsPage />} />
    <Route path="publishers" element={<PublishersPage />} />
    <Route path="bookshelves" element={<BookshelvesPage />} />
    <Route path="import" element={<ImportPage />} />
    <Route path="settings" element={<SettingsPage />} />   { /* Change password */ }
  </Route>
</Routes>
```

Layouts

- **PublicLayout:** Minimal header with site name/logo, search bar, and a small “Admin” link (bottom of page or in header dropdown). Clean, reader-focused.
- **AdminLayout:** Sidebar navigation (collapsible on mobile) with links to Dashboard, Books, Authors, Publishers, Bookshelves, Import, Settings. Top bar shows admin username and logout.

7. Public Catalog (Viewer Page)

Catalog Page (/)

This is what visitors see. It must be clean, fast, and pleasant.

Hero / Header Section

- Site title: “ My Library” (or customizable name)
- Subtitle showing total public book count: “271 books and counting”
- Prominent search bar (searches title, author, ISBN in real-time with debounce)

Filter Bar (horizontal, collapsible on mobile)

- **Format:** All / Physical / Digital / Audio (pill buttons)
- **Binding:** Dropdown with Paperback, Hardcover, Kindle Edition, etc.

- **Author:** Searchable dropdown / autocomplete
- **Publisher:** Searchable dropdown / autocomplete
- **Bookshelf:** Tag-style filter chips
- **Year Range:** Dual-input or range slider for publication year
- **Sort by:** Title (A-Z), Newest Added, Oldest Published, Most Pages, Price (Low-High, High-Low)
- **Reset Filters** button

Book Grid / List

- Default: **Card grid** (responsive: 4 cols desktop, 2 tablet, 1 mobile)
- Toggle to **list view** (compact rows with more data visible)
- Each **book card** shows:
 - Cover image (or a styled placeholder with book title and author if no cover)
 - Title (bold, primary)
 - Author name
 - Format badge (📖 Physical / 📱 Digital / 🎧 Audio) with color coding
 - Page count
 - Publication year
 - Publisher
- **Pagination** at bottom (page numbers + prev/next, or infinite scroll)

Book Detail Page (/books/:id)

- Full book information:
 - Title, author(s) — primary and additional
 - Publisher
 - ISBN / ISBN13
 - Edition info
 - Format & binding type
 - Number of pages
 - Year published & original publication year
 - Date added to library
 - Bookshelves (as tag chips)
 - Cover image (large)
- **No prices shown to public viewers** — pricing is admin-only data
- “← Back to catalog” link

8. Admin Dashboard

Admin Login Page (/admin/login)

- Simple centered form: username + password + “Log in” button
- Error message on invalid credentials
- Redirect to /admin on success

Admin Books Management (/admin/books)

Same as public catalog but with **full admin powers**:

- **All books shown** (including hidden ones — hidden books have a visual indicator like a dimmed card or a “Hidden” badge)
- **Search & filter**: Same as public + additional filters:
 - Status (To Read / Reading / Read / DNF / On Hold)
 - Visibility (All / Public only / Hidden only)
 - Price range filter
- **Bulk actions toolbar** (appears when books are selected via checkboxes):
 - “Hide from public” / “Show to public”
 - “Delete selected”
 - “Change status” (dropdown)
 - “Change format” (dropdown)
- **Per-book actions** (on each card/row):
 -  Edit button → opens edit form
 -  /  Toggle visibility (quick toggle, no page reload)
 -  Delete (with confirmation dialog)
 - **+ Add Book** button (top right) → navigates to `/admin/books/new`

Book Add/Edit Form (/admin/books/new and /admin/books/:id/edit)

A well-organized form with sections:

Section 1: Basic Information

- Title (text input, required)
- Edition (text input, optional — e.g. “1st Edition”, “Revised 2020”)
- Format (radio buttons: Physical / Digital / Audio)
- Binding (dropdown: Paperback, Hardcover, Mass Market Paperback, Kindle Edition, Unknown)
- Number of Pages (number input)

Section 2: Authors

- Primary Author* (searchable dropdown from existing authors, or type to create new)
- Additional Authors (multi-select, searchable, type to create new)

Section 3: Publishing Details

- Publisher (searchable dropdown, or type to create new)
- ISBN (text input)
- ISBN13 (text input)
- Year Published (number input)
- Original Publication Year (number input)

Section 4: Pricing

- Purchase Price (number input with currency label)
- Market Price / Actual Price (number input with currency label)
- Currency (dropdown, default “SAR”)
- **Calculated Savings**: displayed live as `Purchase Price - Market Price` (read-only, color-coded green if saving, red if overpaid)

Section 5: Status & Organization

- Reading Status (dropdown: To Read, Reading, Read, Did Not Finish, On Hold)

- Date Added (date picker, defaults to today)
- Date Started Reading (date picker, optional)
- Date Finished Reading (date picker, optional)
- Bookshelves (multi-select tag input — select existing or create new)
- Publicly Visible (toggle switch, default ON)

Section 6: Additional

- Cover Image URL (text input, with preview)
- Notes (textarea, admin-only notes)

Form Actions:

- “Save” (primary button)
- “Save & Add Another” (secondary, only on create)
- “Cancel” (goes back)
- On edit: “Delete Book” (danger button with confirmation)

Authors Management (/admin/authors)

- Table: Author Name | Number of Books | Actions (Edit / Delete)
- Search by author name
- Add new author inline or via modal
- Delete blocked if author has books (show warning)

Publishers Management (/admin/publishers)

- Table: Publisher Name | Number of Books | Actions (Edit / Delete)
- Same pattern as Authors

Bookshelves Management (/admin/bookshelves)

- Table: Bookshelf Name | Number of Books | Actions (Edit / Delete)
- Tag-style display

9. Book Data Model & Business Logic

Format Mapping from CSV

The CSV `Binding` column maps to both `format` and `binding` :

CSV Binding Value	format	binding
Paperback	PHYSICAL	PAPERBACK
Mass Market Paperback	PHYSICAL	MASS_MARKET_PAPERBACK
Hardcover	PHYSICAL	HARDCOVER
Kindle Edition	DIGITAL	KINDLE_EDITION
Unknown Binding	PHYSICAL	UNKNOWN

The admin can override `format` manually when adding/editing a book. Audiobooks must be set manually since the CSV doesn't distinguish them.

Savings Calculation

```
// NEVER store savings in the database – always compute on the fly
function calculateSavings(purchasePrice: number | null, marketPrice: number | null): number | null {
  if (purchasePrice === null || marketPrice === null) return null;
  return purchasePrice - marketPrice;
  // Positive = you paid MORE than market (overpaid)
  // Negative = you paid LESS than market (you saved money)
  // NOTE: If the owner prefers "savings" to mean market - purchase (positive = saved),
  //       then reverse: return marketPrice - purchasePrice;
}
```

Important: In the CSV data, “Actual Price” is empty for all books (all NaN). The system should still support it for future use when the admin fills in market prices. Show “N/A” or a dash when market price is missing.

Reading Status Rules

Status	When to use	Date fields
TO_READ	Default for new/imported books	Only <code>dateAdded</code>
READING	Currently reading	<code>dateStartedReading</code> set
READ	Finished	Both <code>dateStartedReading</code> and <code>dateFinishedReading</code> set
DID_NOT_FINISH	Abandoned	<code>dateStartedReading</code> set
ON_HOLD	Paused for now	<code>dateStartedReading</code> set

10. CSV Import Feature

Import Page (`/admin/import`)

A dedicated import flow for bulk-adding books from CSV files.

Step 1: Upload

- Drag-and-drop zone or file picker for `.csv` files
- File size limit: 5MB
- Show file name and size after selection

Step 2: Preview & Mapping

- Parse the CSV client-side (using PapaParse) and show a **preview table** of the first 10 rows
- Show **column mapping** — auto-detect columns from the CSV header:

CSV Column	Maps to DB Field
Book Id	externalId
Title	title
Purchase Price	purchasePrice
Actual Price	marketPrice
Author	author.name (find or create)
Additional Authors	additionalAuthors (split by , , find or create each)
ISBN	isbn
ISBN13	isbn13 (clean and format as string)
Publisher	publisher.name (find or create)
Binding	binding + format (see mapping table above)
Number of Pages	numberOfPages
Year Published	yearPublished
Original Publication Year	originalPublicationYear
Date Added	dateAdded (parse from YY-MM-DD format)
Bookshelves	bookshelves (split by , , find or create each)

- Allow the user to **adjust column mappings** via dropdowns if auto-detection fails
- Show validation errors inline (e.g., missing titles, invalid numbers)

Step 3: Import Settings

- **Duplicate handling:** Dropdown with options:
 - Skip duplicates (match on `externalId` or `title + author`)
 - Overwrite duplicates
 - Import all (allow duplicates)
- **Default format:** Choose default BookFormat for imported books (default: PHYSICAL)
- **Default visibility:** Public / Hidden (default: Public)
- **Default status:** Reading status for all imported books (default: TO_READ)

Step 4: Execute & Report

- Send CSV to `POST /api/admin/import/csv` with settings
- Show a progress indicator
- After completion, show a **summary report**:

-  X books imported successfully
-  X books skipped (duplicates)
-  X books failed (with reasons)
- List of new authors/publishers created

CSV Parsing Notes

Based on the actual CSV data:

- **Date format:** YY-MM-DD (e.g., 24-05-26 = May 26, 2024). Parse accordingly.
 - **ISBN13:** Comes as a float (e.g., 9.78614e+12). Convert to string properly: parse as float, convert to integer, then to string.
 - **ISBN:** Comes as either a number or string. Always store as string, trim whitespace.
 - **Additional Authors:** Comma-separated string. May contain the original author name repeated in Arabic. Split by , and trim each.
 - **Bookshelves:** Comma-separated tags like "to-read, importation". Split by , and trim.
 - **NaN / empty values:** Handle gracefully — set to null in the DB.
-

11. Statistics & Analytics (Admin)

Dashboard Page (/admin)

The admin dashboard is the **most important admin page**. It should be a single scrollable page packed with useful statistics, organized into clear sections with cards and charts.

Section 1: Overview KPI Cards (top row)

A horizontal row of **summary cards** with large numbers and icons:

Card	Value	Icon
Total Books	Count of all books	
Total Pages	Sum of all page counts	
Total Spent	Sum of all purchase prices	
Total Savings	Sum of all (market - purchase) where both exist	
Average Book Price	Mean purchase price	
Authors	Count of unique authors	
Publishers	Count of unique publishers	
Books Read	Count where status = READ	
Public Books	Count of publicly visible books	
Hidden Books	Count of hidden books	

Each card should also show a **small secondary value** where useful:

- Total Books → “+X this month”
- Total Pages → “avg X pages/book”
- Total Spent → “avg X per book”

Section 2: Reading Status Breakdown

- **Donut / Pie Chart:** Books by reading status (To Read, Reading, Read, DNF, On Hold)
- **Counts and percentages** next to/below the chart
- Color-coded to match status badges used elsewhere in the app

Section 3: Format Distribution

- **Pie Chart or Horizontal Bar Chart:** Books by format (Physical / Digital / Audio)
- Show count and percentage for each
- Also show **spending per format** as a grouped/stacked bar

Section 4: Spending Analytics

- **Bar Chart:** Total spending per month/year (based on `dateAdded`)
- **Line Chart:** Cumulative spending over time
- **Bar Chart:** Average price per format (Physical vs Digital vs Audio)

- **Table:** Top 10 most expensive books (title, author, purchase price)
 - **Table:** Top 10 cheapest books
 - **Stat Card:** Most expensive book, cheapest book, median price
-

Section 5: Author Analytics

- **Bar Chart (horizontal):** Top 15 authors by number of books
 - **Bar Chart:** Top 10 authors by total spending
 - **Table:** All authors with columns: Name | Book Count | Total Pages | Total Spent | Avg Price
 - Sortable columns
-

Section 6: Publisher Analytics

- **Bar Chart (horizontal):** Top 15 publishers by number of books
 - **Table:** All publishers with columns: Name | Book Count | Total Spent | Avg Price
 - Sortable columns
-

Section 7: Page Count Analytics

- **Histogram:** Distribution of books by page count (buckets: 0-100, 100-200, ..., 1000+)
 - **Stat Cards:** Longest book (title + pages), shortest book, average pages, median pages
 - **Scatter Plot:** Pages vs. Purchase Price (to see if price correlates with length)
-

Section 8: Timeline & Trends

- **Line/Area Chart:** Number of books added per month over time (based on `dateAdded`)
 - **Stacked Area Chart:** Books added by format over time
 - **Bar Chart:** Books by original publication year (decade buckets: 1800s, 1900-1950, 1950-1970, 1970-1990, 1990-2000, 2000-2010, 2010-2020, 2020+)
 - **Stat Card:** Oldest book in collection (by `originalPublicationYear`), newest book
-

Section 9: Bookshelf / Tag Analytics

- **Bar Chart:** Number of books per bookshelf
 - **Tag Cloud:** Visual representation of bookshelves (larger = more books)
-

Section 10: Binding Type Analytics

- **Donut Chart:** Books by binding type (Paperback, Hardcover, Kindle Edition, Mass Market Paperback, Unknown)
 - Counts and percentages
-

Section 11: Quick Lists (Useful Reference Tables)

- **Recently Added Books** (last 10, with date added)
 - **Currently Reading** (all books with status READING)
 - **Books Without Market Price** (count and list — nudge admin to fill them in)
 - **Books Without Page Count** (count and list)
 - **Books Without Cover Image** (count)
 - **Duplicate Authors Check** (authors with very similar names — possible duplicates to merge)
-

Stats API Response Examples

GET /api/admin/stats/overview

```
{
  "totalBooks": 271,
  "totalPages": 105432,
  "totalSpent": 10543.00,
  "totalSavings": null,
  "averagePrice": 38.90,
  "medianPrice": 35.00,
  "totalAuthors": 159,
  "totalPublishers": 86,
  "byStatus": {
    "TO_READ": 250,
    "READING": 5,
    "READ": 10,
    "DID_NOT_FINISH": 3,
    "ON_HOLD": 3
  },
  "byFormat": {
    "PHYSICAL": 250,
    "DIGITAL": 11,
    "AUDIO": 10
  },
  "publicBooks": 260,
  "hiddenBooks": 11,
  "booksAddedThisMonth": 15
}
```

12. UI/UX Design Guidelines

Color Palette

Use a **warm, bookish theme**:

Token	Color	Usage
--primary	#8B5E3C (warm brown)	Buttons, links, active states
--primary-light	#C4956A	Hover states, subtle accents
--secondary	#2C5F2D (forest green)	Success indicators, savings
--background	#FAFAF5 (warm off-white)	Page background
--surface	#FFFFFF	Cards, modals
--text	#1A1A1A	Primary text
--text-muted	#6B7280	Secondary text, placeholders
--danger	#DC2626	Delete buttons, overpaid indicators
--warning	#F59E0B	Warnings, “on hold” status
--info	#3B82F6	Info badges, “reading” status

Format Color Coding

Format	Color	Badge Style
Physical 📖	#8B5E3C brown	Solid brown pill
Digital 📱	#3B82F6 blue	Solid blue pill
Audio 🎧	#8B5CF6 purple	Solid purple pill

Status Color Coding

Status	Color	Badge
To Read	#6B7280 gray	Outlined gray
Reading	#3B82F6 blue	Solid blue
Read	#2C5F2D green	Solid green
Did Not Finish	#DC2626 red	Outlined red
On Hold	#F59E0B yellow	Outlined yellow

Typography

- **Headings:** Inter or IBM Plex Sans (clean, modern)

- **Body text & Arabic content:** Noto Sans Arabic (full Arabic/Latin support)
- Ensure **RTL support** — many book titles and author names are in Arabic. Use `dir="auto"` on text elements displaying book data, or detect and apply RTL dynamically.

Responsive Breakpoints

Breakpoint	Layout
< 640px (mobile)	Single column, hamburger menu for admin sidebar
640px - 1024px (tablet)	2-column book grid, collapsible sidebar
> 1024px (desktop)	3-4 column book grid, full sidebar

Accessibility

- All interactive elements keyboard-navigable
- Proper `aria-labels` on icon-only buttons
- Sufficient color contrast ratios (WCAG AA minimum)
- Focus rings on interactive elements

13. Railway Deployment Guide

Repository Structure for Railway

Railway can deploy from a **monorepo** with two services.

`railway.toml` (root)

```
[build]
builder = "nixpacks"

[deploy]
numReplicas = 1
healthcheckPath = "/api/health"
healthcheckTimeout = 300
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 10
```

Railway Services Setup

Create **3 services** in a Railway project:

1. PostgreSQL Database

- Add a PostgreSQL plugin from Railway's dashboard
- Railway auto-provides `DATABASE_URL` env var

2. Backend (Express API)

- **Root directory:** `server`
- **Build command:**
`npm install && npx prisma generate && npx prisma migrate deploy && npm run build`

- **Start command:** `npm run start`
- **Environment variables:**

```

DATABASE_URL=${{Postgres.DATABASE_URL}}
JWT_SECRET=<generate-a-strong-random-string>
ADMIN_USERNAME=admin
ADMIN_PASSWORD=<your-secure-password>
NODE_ENV=production
PORT=3000
CLIENT_URL=https://<your-frontend-domain>.up.railway.app

```

3. Frontend (React / Vite)

- **Root directory:** `client`
- **Build command:** `npm install && npm run build`
- **Start command:** `npx serve dist -s -l 3000` (or use Vite preview)
- **Environment variables:**

```

VITE_API_URL=https://<your-backend-domain>.up.railway.app/api

```

Database Seeding

Add a **seed script** in `server/prisma/seed.ts` :

```

// 1. Create the admin user (hash the password from env)
// 2. Optionally import the initial CSV data
// Run with: npx prisma db seed

```

Configure in `server/package.json` :

```

{
  "prisma": {
    "seed": "ts-node prisma/seed.ts"
  }
}

```

Health Check Endpoint

Add to the Express server:

```

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

```

CORS Configuration

```

app.use(cors({
  origin: process.env.CLIENT_URL || 'http://localhost:5173',
  credentials: true,
}));

```

Production Checklist

- [] `NODE_ENV=production` set
- [] `JWT_SECRET` is a strong random string (32+ characters)

- [] ADMIN_PASSWORD is strong
- [] CORS origin set to the correct frontend URL
- [] Prisma migrations applied (`prisma migrate deploy`)
- [] Admin user seeded
- [] Rate limiting enabled on auth routes
- [] Helmet security headers enabled

14. Seed Data Reference

The attached CSV (`Book1.csv`) contains **271 books** with this profile:

Metric	Value
Total books	271
Unique authors	159
Unique publishers	86
Price range	10 - 198 SAR
Page count range	~100 - 1,452
Binding types	Paperback (245), Kindle Edition (11), Hardcover (3), Unknown Binding (3), Mass Market Paperback (2)
Bookshelf tags	to-read , importation , روايات , أدب , علم - نفس
Date added range	2021 - 2026
Publication years	1846 - 2025
Market prices	All empty (to be filled later)
Language of content	Mostly Arabic titles and Arabic-translated authors

CSV Column ↔ Schema Quick Reference

CSV Header	Prisma Field	Notes
Book Id	externalId	Goodreads ID, store as string
Title	title	Arabic text, ensure UTF-8
Purchase Price	purchasePrice	Integer in CSV, store as Decimal
Actual Price	marketPrice	All NaN in current data
Author	→ Author.name	Find or create
Additional Authors	→ BookAdditionalAuthor	Comma-separated, find or create each
ISBN	isbn	May be number or string
ISBN13	isbn13	Scientific notation float → convert to 13-digit string
Publisher	→ Publisher.name	Find or create
Binding	binding + format	See mapping in Section 9
Number of Pages	numberOfPages	Float in CSV → parse as integer
Year Published	yearPublished	Float → integer
Original Publication Year	originalPublicationYear	Float → integer
Date Added	dateAdded	Format: YY-MM-DD
Bookshelves	→ BookBookshelf	Comma-separated tags

15. Future Enhancements (Optional)

These are out of scope for the initial build but documented for future reference:

Feature	Description
Cover image auto-fetch	Use Google Books API or Open Library API to auto-fetch cover images by ISBN
Reading goal tracker	Set yearly reading goals (e.g., “Read 50 books in 2025”) with progress bar
Book ratings & reviews	Admin can rate books 1-5 stars and write reviews
Wishlist	Separate section for books the admin wants to buy
Export	Export library as CSV, JSON, or PDF report
Multi-currency support	Convert prices to different currencies using exchange rates
Dark mode	Toggle between light and dark themes
PWA support	Installable on mobile devices with offline access
Barcode scanner	Scan ISBN barcodes with phone camera to add books
Public stats page	A read-only version of some stats (e.g., total books, reading progress) for visitors
Notifications / Reminders	Email reminders for books in “Reading” status for too long
AI recommendations	Based on reading history, suggest similar books
Multi-user support	Allow multiple library owners with separate accounts
Goodreads sync	Periodic import from Goodreads API

Summary of What to Build

Priority	Feature	Status
 P0	PostgreSQL schema with Prisma + migrations	Must have
 P0	Express REST API with all CRUD endpoints	Must have
 P0	Admin authentication (JWT)	Must have
 P0	Public catalog page with search & filters	Must have
 P0	Admin book management (CRUD + visibility toggle)	Must have
 P0	CSV import with column mapping	Must have
 P1	Full statistics dashboard (all 11 sections)	Must have
 P1	Admin authors/publishers/bookshelves management	Must have
 P1	Responsive design + Arabic/RTL support	Must have
 P2	Bulk actions (multi-select visibility/delete)	Nice to have
 P2	Book cover image URL support	Nice to have
 P2	Duplicate detection on import	Nice to have
 P3	Dark mode, PWA, export	Future

This specification is complete and ready for implementation. Feed this entire file to Cursor as context alongside the CSV data file, and instruct it to implement feature-by-feature following the priority order above.